Operating Systems - Unit 2: Processes

A process is a program in execution. A program is a passive entity stored on disk, while a process is an active entity with a program counter, registers, stack and heap.

Process states: New, Ready, Running, Waiting and Terminated. The operating system moves a process between these states as it is scheduled, blocks for I/O, or finishes.

The Process Control Block (PCB) stores everything the OS needs to know about a process: process state, program counter, CPU registers, scheduling information, memory-management information and I/O status.

CPU Scheduling

The CPU scheduler selects a process from the ready queue to run next.

First-Come, First-Served (FCFS): processes run in arrival order. Simple, but short jobs can wait behind long ones (the convoy effect).

Shortest Job First (SJF): the process with the smallest next CPU burst runs first. It gives the minimum average waiting time but needs burst lengths to be known or predicted.

Round Robin (RR): each process gets a fixed time quantum, for example 10 ms. When the quantum expires the process is pre-empted and moved to the back of the ready queue. A very large quantum makes RR behave like FCFS; a very small one causes too many context switches.

Priority scheduling runs the highest-priority process first. Low-priority processes can starve; ageing gradually raises the priority of waiting processes.

Deadlock

A deadlock is a situation where a set of processes are blocked because each one holds a resource and waits for a resource held by another.

Four necessary conditions (Coffman conditions): mutual exclusion, hold and wait, no pre-emption, and circular wait. All four must hold at the same time for deadlock to occur.

Deadlock prevention breaks one of the four conditions. Deadlock avoidance, such as the Banker's algorithm, only grants a request if the system stays in a safe state. Starvation is different from deadlock: a starving process could run, but is repeatedly passed over.